Universidade Católica de Brasília

Pró-Reitoria Acadêmica
Curso de Engenharia de Software
Laboratório de Banco de Dados

RELATÓRIO FINAL

Docente: Samuel Novais.
Discentes: Maria Eduarda Gomes da Silva, Sarah Cruz Serpa, Victor Gabriel Andrade, Vitor Lustosa Leão Pinheiro.
Turma: Engenharia de Software, GPE17M40079.
Versão MySQL 8.0.46

ETAPA 1 - Projeto e construção do banco

1 Descrição do domínio

O sistema PetCare foi idealizado para gerenciar os processos operacionais, clínicos e administrativos de Organizações Não Governamentais (ONGs) voltadas ao acolhimento, cuidado e adoção de animais em situação de vulnerabilidade. O domínio modelado abrange desde o resgate e abrigo do animal até o seu encaminhamento para um novo lar, integrando a gestão de parceiros financeiros, aplicação de vacinas, o acompanhamento clínico veterinário e o andamento de pedidos de adoção.

No sistema estão as ONGs, entidades jurídicas cadastradas no sistema por meio de seu CNPJ, responsáveis por abrigar os animais resgatados. Cada animal possui características próprias (espécie, raça, sexo e data de nascimento) e mantém um histórico de sua Situação, registrando datas de chegada, motivos de acolhimento, status atuais e eventuais datas de saída. Para manter a transparência financeira e operacional, as ONGs recebem suporte de Parceiros, que registram patrocínios contendo valores, datas e tipos de apoio fornecidos. A administração interna da ONG é realizada por usuários específicos do tipo administrativo (admOng).

O cuidado com a saúde dos animais é uma frente fundamental do domínio. Todo animal possui um Prontuário exclusivo que centraliza seu histórico médico. Os atendimentos clínicos são realizados por Veterinários que são profissionais cadastrados obrigatoriamente com um número de CRMV válido, que podem estabelecer relações hierárquicas de supervisão entre si. O prontuário registra as Consultas veterinárias (com diagnósticos e notas), a Aplicação de Vacinas (detalhando doses, lotes e datas a partir do catálogo de vacinas disponíveis) e a Solicitação de Exames baseada em um catálogo de TipoExame, controlando datas de solicitação, realização e resultados.

Outro processo central do PetCare é o fluxo de adoção. Pessoas cadastradas como Adotantes podem fazer pedidos de adoção para animais disponíveis. Cada pedidoAdocao é submetido à avaliação dos administradores da ONG (admOng), que registram a data da decisão, notas e o parecer final (aprovado ou recusado). O sistema assegura que o animal esteja com status adequado para adoção e mantém o controle sobre o histórico do adotante.

Por fim, o modelo gerencia de forma unificada as Pessoas do sistema por meio de uma generalização/especialização, distinguindo os papéis de adotantes, veterinários, administradores de ONG e administradores do sistema (AdmSistema), garantindo a unicidade de documentos como CPF e CRMV e assegurando que todas as operações respeitem a integridade estrutural do negócio.

1.1 Regras de negócio

As regras abaixo são afirmações verificáveis sobre o domínio, numeradas de RN01 em diante, conforme exigido. A coluna "Atendida por" indica se a regra será garantida por restrição do banco, por uma consulta, ou pela camada de aplicação.

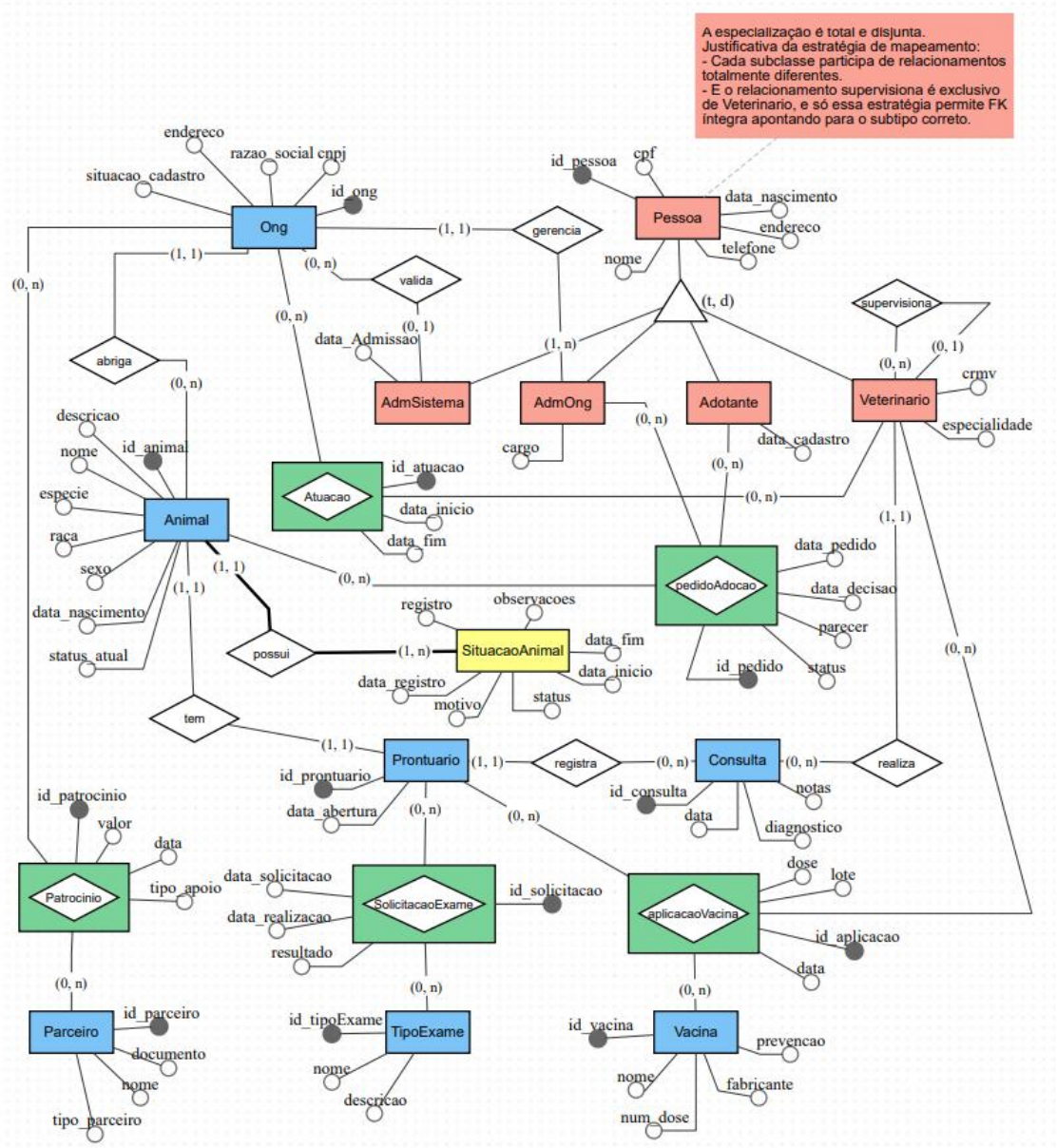
Código	Regra de negócio	Atendida por
RN01	Um animal só pode ser vinculado a uma ONG (abriga) se a ONG já estiver devidamente cadastrada no sistema.	Restrição do Banco (FK fk_animal_ong)
RN02	Todo animal cadastrado precisa estar vinculado a uma ONG responsável.	Restrição do Banco (NOT NULL em Animal.id_ong)
RN03	Uma ONG pode ser gerenciada por vários administradores (admOng), mas cada administrador cuida de uma ONG específica	Restrição do Banco (Chave Estrangeira)
RN04	Todo registro de patrocínio deve conter obrigatoriamente o valor, a data e o tipo de apoio oferecido pelo parceiro.	Restrição do Banco (NOT NULL)
RN05	Um parceiro só pode patrocinar uma ONG após ambos os cadastros (Parceiro e ONG) estarem cadastrados no sistema.	Restrição do Banco (FKs fk_patrocinio_ong, fk_patrocinio_parceiro)
RN06	Cada pedido de adoção pode ser avaliado por no máximo um administrador da ONG por vez.	Restrição do Banco (coluna única id_adm_ong, FK opcional fk_pedido_admong)
RN07	Um adotante só pode ter um pedido com status "Em análise" por vez; para pedir outro animal, o anterior deve estar Aprovado, Recusado ou Cancelado.	Camada de Aplicação / Consulta
RN08	Se um pedido de adoção for recusado, o motivo ou parecer da recusa deve ser obrigatoriamente preenchido.	Restrição do Banco (CHECK ck_pedido_parecer_se_recusado)
RN09	O sistema deve preencher automaticamente a data de decisão sempre que um pedido de adoção for aprovado ou recusado	Camada de Aplicação (gravada na mesma operação que altera o status)

RN10	Um pedido só pode ser aprovado se o animal estiver "Disponível" ou "Em Adoção" no momento da decisão; animais em "Tratamento", "Adotado" ou "Óbito" não podem ser adotados.	Camada de Aplicação / Consulta
RN11	Todo animal deve receber um prontuário logo no momento em que é cadastrado.	Camada de Aplicação (Transação de Cadastro)
RN12	Qualquer mudança na situação de um animal (como chegada, alteração de status ou saída) gera um novo histórico na tabela SituacaoAnimal	Camada de Aplicação (gravada na mesma operação que altera o status)
RN13	Nenhum registro de histórico em SituacaoAnimal pode existir solto, sem estar ligado a um animal.	Restrição do Banco (Chave Estrangeira)
RN14	Uma consulta veterinária deve estar vinculada a um veterinário responsável e a um prontuário.	Restrição do Banco (Chaves Estrangeiras)
RN15	Uma aplicação de vacina deve registrar obrigatoriamente a dose e o lote utilizados, além da data de aplicação.	Restrição do Banco (NOT NULL)
RN16	Uma vacina só pode ser aplicada por um veterinário cadastrado, identificado por CRMV.	Restrição do Banco (FK fk_aplicvac_veterinario; CRMV NOT NULL)
RN17	CPF de pessoa, documento de parceiro, CNPJ de ONG e CRMV de veterinário não podem se repetir no sistema.	Restrição do Banco (UNIQUE)
RN18	Todo exame solicitado para um animal deve estar vinculado a um tipo de exame cadastrado (TipoExame) e a um prontuário, registrando as datas de solicitação e realização.	Restrição do Banco (Chaves Estrangeiras)
RN19	As datas de nascimento do animal, da consulta, da aplicação de vacina e da solicitação de exame não podem ser futuras; a data de realização do exame só pode ser futura enquanto ele estiver pendente (sem resultado).	Restrição do Banco (gatilhos trg_*_check_data*)
RN20	Todo veterinário pode ter um colega supervisor registrado para acompanhar suas atividades.	Restrição do Banco (FK autorreferente opcional fk_veterinario_supervisor + gatilhos trg_veterinario_check_supervisor_*)
RN21	Somente o administrador da ONG aprova ou recusa pedidos de adoção.	Aplicação (controle de acesso/perfil)
RN22	Somente veterinário com vínculo vigente	Aplicação (controle de

	(Atuacao) com a ONG do animal registra e edita o prontuário.	acesso)
RN23	Animal adotado não é excluído do sistema; permanece cadastrado com status "Adotado" para rastreabilidade.	Aplicação (exclusão lógica)
RN24	ONG só publica animais para adoção após validação institucional pelo administrador do sistema.	Aplicação (restrição de processo)
RN25	Dados pessoais do adotante só são visíveis ao administrador da ONG responsável pelo processo em andamento.	Aplicação (controle de acesso)
RN26	Sexo do animal, status do animal, situação de cadastro da ONG, status no do pedido e tipo de parceiro só assumem os valores das listas fechadas do dicionário de dados.	Aplicação (validação no cadastro)
RN27	Um veterinário pode atuar em mais de uma ONG ao longo do tempo; cada atuação tem data de início e, quando encerrada, data de fim (vazia enquanto vigente).	Restrição do Banco (FKs fk_atuacao_ong, fk_atuacao_veterinario; NOT NULL em data_inicio)
RN28	Alterações de situação de um animal e da situação, do parecer ou do avaliador de um pedido de adoção são registradas com autor, data e valores anterior e posterior.	Restrição do banco (gatilhos trg_animal_audita_update e trg_pedido_audita_update)

(Alteração na Etapa 2: Adição da RN28)

2. Modelo Entidade Relacionamento conceitual



Generalização/especialização

A tabela **Pessoa** foi dividida em quatro tipos: **Adotante**, **Veterinario**, **AdmOng** e **AdmSistema**, representando diferentes papéis que uma pessoa física exerce no sistema. Toda pessoa pertence a um e apenas um desses papéis, por isso a especialização é total e disjunta.

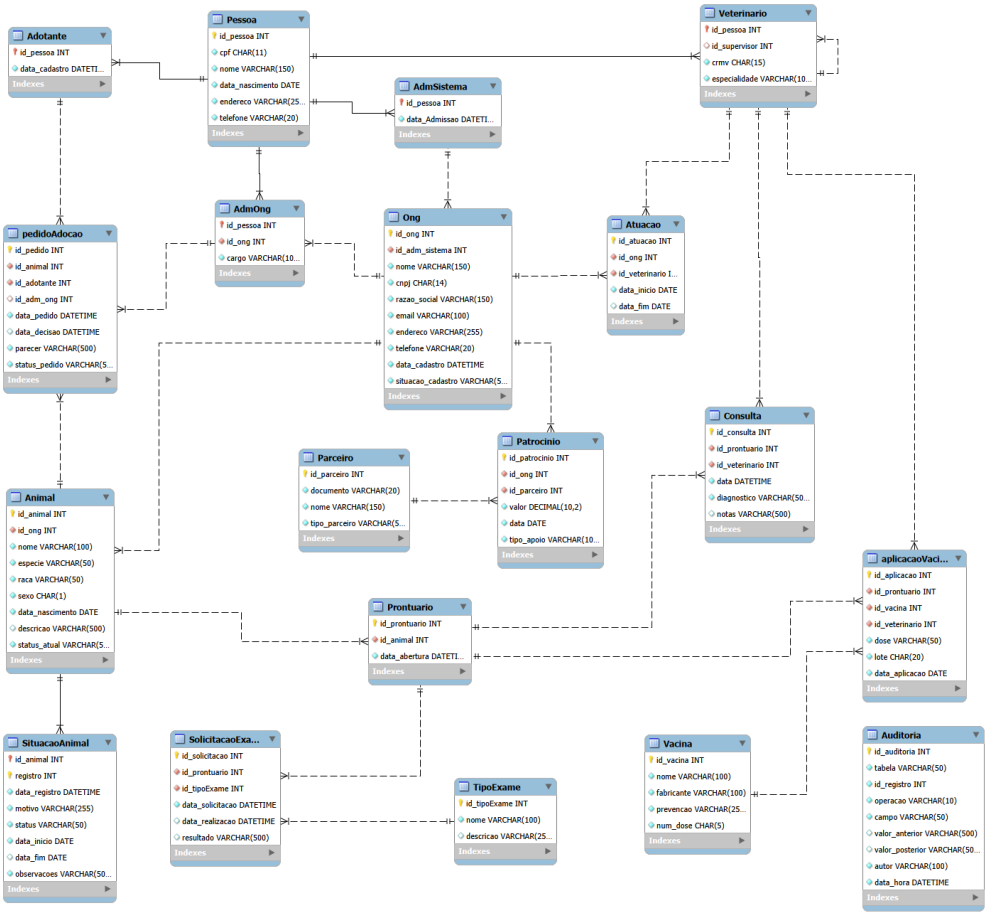
Estratégia de mapeamento: uma tabela para a superclasse Pessoa contendo os atributos comuns e uma tabela para cada subtipo contendo apenas os atributos específicos, ligada por FK/PK compartilhada com Pessoa.id_pessoa.

Justificativa:

- **Garante a unicidade de CPF:** Mantém a regra (RN17) que proíbe CPFs repetidos.
- **Evita campos vazios:** Evita colunas nulas em excesso que ocorreriam em uma tabela única (Ex: crmv só existe para veterinários).
- Permite que cada subtipo mantenha relacionamentos próprios sem sobrecarregar a superclasse. (Ex: Apenas Veterinario participa de Consulta e Atuacao e apenas AdmOng participa da avaliação de PedidoAdocao).
- Facilita a evolução do sistema caso novos papéis sejam adicionados.

3. Modelo lógico relacional

(Alteração na Etapa 2: Adição da tabela Auditoria)



Justificativas:

Especialização: A escolha foi uma tabela para cada subclasse, porque assim:

- Evita o excesso de colunas nulas. Por exemplo: crmv e id_supervisor só fazem sentido para veterinário, id_ong só para adm_ong. Numa tabela única, cada linha teria várias colunas preenchidas com NULL.

- Evita a duplicação de atributos comuns (nome, endereço, telefone, data_nascimento), e preserva a unicidade de CPF exigida pela RN17, com tabelas separadas por subtipo, a mesma pessoa poderia, em tese, ser cadastrada com o mesmo CPF em duas tabelas distintas sem violar nenhuma constraint.

- Mantém a especialização total e separada do conceitual. Cada cpf em pessoa aparece em exatamente uma das quatro tabelas de subtipo.

- Permite que relacionamentos específicos de cada papel (ex.: id_veterinario em consulta e atuacao; id_admOng em pedido_adocao) apontem diretamente para a tabela do subtipo, sem precisar de verificação adicional de “papel” em tempo de consulta.

Relacionamento Animal-Prontuário: Para esse relacionamento que tem participação obrigatória nos dois lados (1,1:1,1), foi a melhor opção deixar a tabelas separadas e na Tabela Prontuario tem uma chave primária substituta pois:

- Os dois possuem domínios distintos. O animal concentra dados cadastrais/administrativos (espécie, ONG responsável, status); já o prontuário concentra dados clínicos.
- Fundir misturaria os dois contextos de acesso diferentes; a RN22 exige vínculo do veterinário com a ONG, só para edição do prontuário, o que é mais natural de aplicar em uma tabela isolada.
- O prontuário é o "ponto de entrada" de todo o histórico clínico (consultas, vacinas, exames). Mantê-lo como entidade própria evita reestruturar o animal caso no futuro seja necessário versionar ou auditar o prontuário separadamente.
- Evita que a tabela animal cresça com colunas raramente usadas em consultas de rotina. Ex: listagens de adoção não precisam tocar em dados de prontuário.

Chave substituta: Foi usada uma chave substituta(id) para todas as tabelas, porque:

- Nem toda entidade tem CPF/CNPJ, como animal, prontuario, vacina, tipo_exame e auditoria.
- CPF é dado pessoal sensível sob a LGPD. Usar como chave primária significa que ele se propaga como chave estrangeira em toda tabela relacionada, e isso traz exposição ao dado.
- Alguns “nomes” podem ser usados como chave natural, mas no futuro pode acontecer de ter que alterar ou editar. Uma chave substituta colabora para não ter que fazer alteração de todos os registros que estiverem ligados a ele.
- Alguns eventos podem se repetir para o mesmo par de entidades relacionadas ao longo do tempo, como por exemplo: o mesmo animal toma a mesma vacina em datas diferentes.

(Alteração da Etapa 2)

Tabela Auditoria: Foi decidido que a tabela auditoria não poderia ter uma chave estrangeira, pois deve sobreviver à exclusão do registro auditado, e uma FK com CASCADE apagaria o histórico. Outro motivo são os campos tabela e id_registro apontam para tabelas diferentes, e uma FK aponta somente para uma tabela.

4. Dicionário de dados conceitual

Pessoa

Tipo: Entidade forte (superclasse) Dados comuns a todas as pessoas físicas do sistema

Atributo	Descrição	Domínio	Obrig.	Observação
id_pessoa	Identificador único, gerado automaticamente (AUTO_INCREMENT).	Inteiro sequencial	Sim	PK
cpf	CPF da pessoa; único no sistema (RN17).	11 dígitos numéricos	Sim	RN17 - não pode repetir
nome	Nome completo da pessoa.	Texto de até 150 caracteres	Sim	—
data_nascimento	Data de nascimento da pessoa.	Data	Sim	—
endereco	Endereço completo.	Texto de até 255 caracteres	Sim	—
telefone	Telefone de contato.	Texto de até 20 caracteres	Sim	—

Parceiro

Tipo: Entidade forte. Empresas ou pessoas jurídicas/físicas que patrocinam ONGs (relacionadas via Patrocinio)

Atributo	Descrição	Domínio	Obrig.	Observação
id_parceiro	Identificador único, gerado automaticamente.	Inteiro sequencial	Sim	Chave primária substituta
documento	CPF ou CNPJ do parceiro; único no sistema (RN17).	11 digitos (CPF) ou 14 digitos (CNPJ)	Sim	RN17 - não pode repetir
nome	Nome ou razão social do parceiro.	Texto de até 150 caracteres	Sim	—
tipo_parceiro	Classificação do parceiro (ex.: empresa, pessoa física, ONG parceira).	Lista fechada: Empresa, Pessoa Física, ONG Parceira	Sim	Controlada pela aplicação

TipoExame

Tipo: Entidade forte Catálogo dos tipos de exame que podem ser solicitados para um animal (relacionada via SolicitacaoExame)

Atributo	Descrição	Domínio	Obrig.	Observação
id_tipoExame	Identificador único, gerado automaticamente.	Inteiro sequencial	Sim	Chave primária substituta
nome	Nome do tipo de exame.	Texto de até 100 caracteres	Sim	—
descricao	Descrição complementar do exame.	Texto de até 255 caracteres	Não	Aceita ausência de descrição

Vacina

Tipo: Entidade forte Catálogo das vacinas disponíveis para aplicação em animais

Atributo	Descrição	Domínio	Obrig.	Observação
id_vacina	Identificador único, gerado automaticamente.	Inteiro sequencial-	Sim	Chave primária substituta
nome	Nome da vacina.	Texto de até 100 caracteres	Sim	—
fabricante	Fabricante da vacina.	Texto de até 100 caracteres	Sim	—

prevencao	Doenças/condições que a vacina previne.	Texto de até 255 caracteres	Sim	—
num_dose	Total de doses do esquema vacinal (ex.: 1, 2, 3)	Texto de até 5 caracteres	Sim	—

Ong

Tipo: Entidade forte ONGs cadastradas na plataforma, responsáveis pelos animais disponibilizados para adoção.

Atributo	Descrição	Domínio	Obrig.	Observação
id_ong	Identificador único, gerado automaticamente.	Inteiro sequencial	Sim	Chave primária substituta
id_adm_sistema	Administrador do sistema que validou a ONG; opcional até a validação (RN24). ON DELETE SET NULL.	Identificador de AdmSistema	Não	FK → AdmSistema.id_pessoa RN24 - nulo até a validação institucional
nome	Nome fantasia da ONG.	Texto de até 150 caracteres	Sim	—
cnpj	CNPJ da ONG; único no sistema.	14 dígitos numéricos	Sim	UNIQUE
razao_social	Razão social da ONG.	Texto de até 150 caracteres	Sim	—
email	E-mail de contato.	e-mail de até 100 caracteres	Sim	—
endereco	Endereço da ONG.	Texto de até 255 caracteres	Sim	—
telefone	Telefone de contato.	Texto de até 20 caracteres	Sim	—
data_cadastro	Data/hora do cadastro da ONG.	Data e hora	Sim	—
situacao_cadastro	Situação do cadastro; padrão 'Em análise' até validação institucional (RN24).	Lista fechada: Em análise, Validado, Recusado	Sim	RN24- permanece “Em análise” até a validação pelo adm do sistema. Controlada pela aplicação

Animal

Tipo: Entidade forte Animais cadastrados por uma ONG e disponibilizados para adoção.

Atributo	Descrição	Domínio	Obrig.	Observação

id_animal	Identificador único, gerado automaticamente.	Inteiro sequencial	Sim	Chave primária substituta
id_ong	ONG responsável pelo animal; obrigatória (RN01, RN02). ON DELETE RESTRICT.	Identificador de Ong	Sim	RN01 e RN02
nome	Nome do animal.	Texto de até 100 caracteres	Sim	—
especie	Espécie do animal (ex: Cachorro, Gato).	Texto de até 50 caracteres	Sim	—
raca	Raça do animal.	Texto de até 50 caracteres	Sim	Coerência entre espécie e raça validada na aplicação
sexo	Sexo do animal ('M' ou 'F').	Um caractere: M ou F	Sim	Controlada pela aplicação
data_nascimento	Data de nascimento; não pode ser futura (RN19, verificado por trigger).	Data	Sim	RN19 - não pode ser futura
descricao	Descrição livre do animal.	Texto de até 500 caracteres	Não	Aceita ausência de descrição
status_atual	Situação atual do animal; padrão 'Tratamento'.	Lista fechada: Tratamento, Disponível, Em Adoção, Adotado, Óbito	Sim	RN12 e RN10 Controlada pela aplicação

Prontuário

Tipo: Entidade forte (dependente) Prontuário único de cada animal, aberto no momento do cadastro (relação 1:1 com Animal, RN11).

Atributo	Descrição	Domínio	Obrig.	Observação
id_prontuario	Identificador único, gerado automaticamente.	Inteiro sequencial	Sim	Chave primária substituta
id_animal	Animal ao qual o prontuário pertence; UNIQUE garante a cardinalidade 1:1 (RN11). ON DELETE CASCADE.	Identificador de Animal	Sim	RN11
data_abertura	Data/hora de abertura do prontuário.	Data e hora	Sim	—

SituacaoAnimal

Tipo: Entidade fraca Histórico de mudanças de situação de um animal; identificada por (id_animal, registro) — não existe sem um animal (RN13) e cada mudança gera um novo registro (RN12).

Atributo	Descrição	Domínio	Obrig.	Observação
id_animal	Animal ao qual o histórico pertence; obrigatório	Identificador de Animal	Sim	RN13

	(RN13). ON DELETE CASCADE.			
registro	Número sequencial do registro por animal (MAX(registro)+1, calculado pela aplicação — RN12).	Inteiro sequencial por animal, iniciando em 1	Sim	RN12
data_registro	Data/hora em que o registro foi criado.	Data e hora	Sim	—
motivo	Motivo da mudança de situação.	Texto de até 255 caracteres	Sim	—
status	Situação do animal neste registro.	Lista fechada: mesmo domínio de Animal.status_atual	Sim	Controlada pela aplicação
data_inicio	Data de início da situação registrada.	Data	Sim	—
data_fim	Data de fim da situação, quando aplicável.	Data	Não	Nula enquanto a situação estiver vigente
observacoes	Observações complementares.	Texto de até 500 caracteres	Não	Aceita ausência de observações

Consulta

Tipo: Entidade forte (dependente) Consultas veterinárias realizadas para um animal, vinculadas obrigatoriamente a um prontuário e a um veterinário (RN14).

Atributo	Descrição	Domínio	Obrig.	Observação
id_consulta	Identificador único, gerado automaticamente.	Inteiro sequencial	Sim	Chave primária substituta
id_prontuario	Prontuário da consulta; obrigatório (RN14). ON DELETE RESTRICT.	Identificador de Prontuário	Sim	RN14
id_veterinario	Veterinário responsável; obrigatório (RN14). ON DELETE RESTRICT.	Identificador de Veterinário	Sim	RN14
data	Data/hora da consulta; não pode ser futura (RN19, verificado por trigger).	Data e hora	Sim	RN19
diagnostico	Diagnóstico registrado na consulta.	Texto de até 500 caracteres	Sim	—
notas	Notas adicionais da consulta.	Texto de até 500 caracteres	Não	Aceita ausência de notas

AdmSistema

Tipo: Subtipo de Pessoa Administradores responsáveis pela validação institucional das ONGs (RN24).

Atributo	Descrição	Domínio	Obrig.	Observação
id_pessoa	Mesma chave de Pessoa (herança por PK/FK). ON DELETE CASCADE.	Identificador de pessoa	Sim	Chave primária e estrangeira (herança)
data_Admissao	Data/hora de admissão do administrador.	Data e hora	Sim	RN24

AdmOng

Tipo: Subtipo de Pessoa Administradores vinculados a uma ONG específica; uma ONG pode ter vários admOng, mas cada admOng cuida de apenas uma ONG (RN03).

Atributo	Descrição	Domínio	Obrig.	Observação
id_pessoa	Mesma chave de Pessoa (herança por PK/FK). ON DELETE CASCADE.	Identificador de pessoa	Sim	Chave primária e estrangeira (herança)
id_ong	ONG à qual o administrador pertence; obrigatório (RN03). ON DELETE RESTRICT.	Identificador de Ong	Sim	RN03
cargo	Cargo do administrador na ONG.	Texto de até 100 caracteres	Sim	—

Adotante

Tipo: Subtipo de Pessoa Pessoas cadastradas para solicitar a adoção de animais.

Atributo	Descrição	Domínio	Obrig.	Observação
id_pessoa	Mesma chave de Pessoa (herança por PK/FK). ON DELETE CASCADE.	Identificador de pessoa	Sim	Chave primária e estrangeira (herança)
data_cadastro	Data/hora do cadastro como adotante.	data e hora	Sim	—

Veterinario

Tipo: Subtipo de Pessoa Veterinários responsáveis por consultas, exames e aplicação de vacinas; pode ter um supervisor (autorrelacionamento opcional, RN20).

Atributo	Descrição	Domínio	Obrig.	Observação
id_pessoa	Mesma chave de Pessoa (herança por PK/FK). ON DELETE CASCADE.	Identificador de pessoa	Sim	Chave primária e estrangeira (herança)
id_supervisor	Supervisor do veterinário; opcional. Não pode ser ele mesmo (RN20, verificado por trigger). ON DELETE SET NULL.	Identificador de Veterinário	Não	RN20
crmv	Registro no CRMV; único, garante que só veterinário registrado aplique vacina/atenda (RN16).	Texto de 15 caracteres	Sim	RN16
especialidade	Especialidade do veterinário.	Texto de até 100 caracteres	Sim	—

Atuacao

Tipo: Associativa: Ong × Veterinario Registra os períodos em que um veterinário atua em uma ONG.

Atributo	Descrição	Domínio	Obrig.	Observação
id_atuacao	Identificador único, gerado automaticamente.	Inteiro sequencial	Sim	Chave primária substituta
id_ong	ONG onde o veterinário atua. ON DELETE CASCADE.	Identificador de ong	Sim	
id_veterinario	Veterinário que atua na ONG. ON DELETE CASCADE.	Identificador de Veterinário	Sim	
data_inicio	Início do período de atuação.	Data	Sim	—
data_fim	Fim do período de atuação.	Data	Não	Nula enquanto o veterinário ainda atua na ONG

Patrocinio

Tipo: Associativa: Ong × Parceiro Registra os patrocínios de um parceiro a uma ONG, com valor, data e tipo de apoio obrigatórios (RN04, RN05).

Atributo	Descrição	Domínio	Obrig.	Observação
id_patrocinio	Identificador único, gerado automaticamente.	Inteiro sequencial	Sim	Chave primária substituta
id_ong	ONG patrocinada; ambos os cadastros devem existir (RN05). ON DELETE RESTRICT.	Identificador de ong	Sim	RN05

id_parceiro	Parceiro patrocinador (RN05). ON DELETE RESTRICT.	Identificador de Parceiro	Sim	RN05
valor	Valor do patrocínio, obrigatório (RN04).	Decimal com até 10 dígitos e 2 casas	Sim	RN04
data	Data do patrocínio, obrigatória (RN04).	Data	Sim	RN04
tipo_apoio	Tipo de apoio prestado, obrigatório (RN04).	Texto de até 100 caracteres	Sim	RN04

SolicitacaoExame

Tipo: Associativa: Prontuario × TipoExame Registra a solicitação e o resultado de um exame para um animal, com datas de solicitação e realização (RN18).

Atributo	Descrição	Domínio	Obrig.	Observação
id_solicitacao	Identificador único, gerado automaticamente.	Inteiro sequencial	Sim	Chave primária substituta
id_prontuario	Prontuário do animal examinado (RN18). ON DELETE RESTRICT.	Identificador de Prontuário	Sim	RN18
id_tipoExame	Tipo de exame solicitado (RN18). ON DELETE RESTRICT.	Identificador de TipoExame	Sim	RN18
data_solicitacao	Data/hora da solicitação; nunca futura (RN18, RN19, verificado por trigger).	Data e hora	Sim	RN18 e RN19
data_realizacao	Data/hora da realização; NULL enquanto pendente/agendado. Deve ser válida e não futura quando há resultado (RN18, RN19, trigger).	Data e hora	Não	RN18
resultado	Resultado do exame, quando já realizado.	Texto de até 500 caracteres	Não	Preenchido somente após a realização

AplicacaoVacina

Tipo: Associativa: Prontuario × Vacina × Veterinario Registra a aplicação de uma vacina em um animal, com dose, lote e data obrigatórios (RN15), aplicada apenas por veterinário cadastrado (RN16).

Atributo	Descrição	Domínio	Obrig.	Observação
id_aplicacao	Identificador único, gerado automaticamente.	Inteiro sequencial	Sim	Chave primária substituta
id_prontuario	Prontuário do animal vacinado. ON DELETE RESTRICT.	Identificador de Prontuário	Sim	

id_vacina	Vacina aplicada. ON DELETE RESTRICT.	Identificador de vacina	Sim	
id_veterinario	Veterinário responsável pela aplicação; obrigatório (RN16). ON DELETE RESTRICT.	Identificador de Veterinário	Sim	RN16
dose	Dose aplicada, obrigatória (RN15).	Texto de até 50 caracteres	Sim	RN15
lote	Lote da vacina, obrigatório (RN15).	Texto de 20 caracteres	Sim	RN15
data_aplicacao	Data da aplicação; obrigatória e não pode ser futura (RN15, RN19, trigger).	Data	Sim	RN15 e RN19

PedidoAdocao

Tipo: Associativa: Animal × Adotante × AdmOng Registra os pedidos de adoção de um animal por um adotante e sua avaliação por um administrador de ONG (RN06 a RN09).

Atributo	Descrição	Domínio	Obrig.	Observação
id_pedido	Identificador único, gerado automaticamente.	Inteiro sequencial	Sim	Chave primária substituta
id_animal	Animal solicitado para adoção. ON DELETE RESTRICT.	Identificador de Animal	Sim	—
id_adotante	Adotante solicitante; apenas 1 pedido ativo por adotante, controlado pela aplicação (RN07). ON DELETE RESTRICT.	Identificador de Adotante	Sim	RN07
id_adm_ong	Administrador que avalia o pedido; opcional até a avaliação, no máximo um por vez (RN06). ON DELETE SET NULL.	Identificador de AdmOng	Não	RN06
data_pedido	Data/hora do pedido de adoção.	Data e Hora	Sim	—
data_decisao	Data/hora da decisão, preenchida quando aprovado ou recusado (RN09).	Data e hora	Não	RN09
parecer	Justificativa da decisão; obrigatória quando o pedido é recusado (RN08, garantido por CHECK).	Texto de até 500 caracteres	Não	RN08
status_pedido	Status do pedido; padrão 'Em análise'.	Lista fechada: Em análise, Aprovado, Recusado e Cancelado	Sim	RN10 Controlada pela aplicação

(Alteração na Etapa 2)

índices Auxiliares

Criados para otimizar buscas por chave estrangeira frequentemente consultadas:

Índice	Colunas	Finalidade
idx_animal_ong	Animal(id_ong).	Busca de animais por ONG
idx_consulta_veterinario	Consulta(id_veterinario)	Busca de consultas por veterinário
idx_pedido_animal	pedidoAdocao(id_animal)	Busca de pedidos por animal
idx_pedido_adotante	pedidoAdocao(id_adotante)	Busca de pedidos por adotante

Auditoria

Tabela técnica, registro de qualquer alteração de registros das tabelas Animal e pedidoAdoção.

Atributo	Descrição	Domínio	Obrig.	Observação
id_auditoria	Identificador do registro de auditoria	Inteiro sequencial	Sim	Chave primária substituta
tabela	Tabela onde ocorreu a alteração	Texto de até 50 caracteres	Sim	RN28
id_registro	Chave primária do registro alterado	Inteiro	Sim	Sem FK, por ser genérica
operacao	Tipo de operação	Lista fechada: INSERT, UPDATE, DELETE	Sim	CHECK ck_auditoria_operacao
campo	Coluna alterada	Texto de até 50 caracteres	Sim	Uma linha para cada campo alterado
valor_anterior	Valor antes da alteração	Texto de até 500 caracteres	Não	Pode ser nulo, caso o campo esteja vazio
valor_posterior	Valor depois da alteração	Texto de até 500 caracteres	Não	Pode ser nulo, caso o campo esteja vazio
autor	Quem fez a alteração	Texto de até 100 caracteres	Sim	Informado pela aplicação, neste caso
data_hora	Momento da alteração	Data e hora	Sim	Preenchida automaticamente

5. Verificação de normalização

Relação	Chaves candidatas	Dependências	Forma Normal
Pessoa	id_pessoa; cpf	id_pessoa → cpf, nome, data_nascimento, endereco, telefone cpf → id_pessoa (e demais atributos)	BCNF
Parceiro	id_parceiro; documento	id_parceiro → documento, nome, tipo_parceiro documento → id_parceiro (e demais)	BCNF
TipoExame	id_tipoExame	id_tipoExame → nome, descricao	BCNF
Vacina	id_vacina	id_vacina → nome, fabricante, prevencao, num_dose	BCNF
Ong	id_ong; cnpj	id_ong → id_adm_sistema, nome, cnpj, razao_social, email, endereco, telefone, data_cadastro, situacao_cadastro cnpj → id_ong (e demais)	BCNF
AdmSistema	id_pessoa	id_pessoa → data_Admissao	BCNF
AdmOng	id_pessoa	id_pessoa → id_ong, cargo	BCNF
Adotante	id_pessoa	id_pessoa → data_cadastro	BCNF
Veterinario	id_pessoa; crmv	id_pessoa → id_supervisor, crmv, especialidade crmv → id_pessoa (e demais)	BCNF
Animal	id_animal	id_animal → id_ong, nome, especie, raca, sexo, data_nascimento, descricao, status_atual	BCNF (ver 5.1 e 5.2)
Prontuario	id_prontuario; id_animal	id_prontuario → id_animal, data_abertura id_animal → id_prontuario, data_abertura (UNIQUE, relação 1:1)	BCNF
SituacaoAnimal	(id_animal, registro)	(id_animal, registro) → data_registro, motivo, status, data_inicio, data_fim, observacoes	BCNF
Consulta	id_consulta	id_consulta → id_prontuario, id_veterinario, data, diagnostico, notas	BCNF
Atuacao	id_atuacao	id_atuacao → id_ong, id_veterinario, data_inicio, data_fim	BCNF
Patrocinio	id_patrocinio	id_patrocinio → id_ong, id_parceiro, valor, data, tipo_apoio	BCNF
SolicitacaoExame	id_solicitacao	id_solicitacao → id_prontuario, id_tipoExame, data_solicitacao, data_realizacao, resultado	BCNF

AplicacaoVacina	id_aplicacao	id_aplicacao → id_prontuario, id_vacina, id_veterinario, dose, lote, data_aplicacao	BCNF
PedidoAdocao	id_pedido	id_pedido → id_animal, id_adotante, id_adm_ong, data_pedido, data_decisao, parecer, status_pedido	BCNF
Audtoria	id_auditoria	id_auditoria → tabela, id_registro, operacao, campo, valor_anterior, valor_posterior, autor, data_hora	BCNF

- 1FN: todos os atributos são atômicos. Endereço e telefone são tratados como texto único, e não há atributos multivalorados nem grupos repetitivos.
- 2FN: as únicas chaves compostas são as de SituacaoAnimal, e todos os seus atributos dependem da chave inteira, pois “registro” só faz sentido dentro de um animal.
- 3FN e BCNF: em todas as relações, todo determinante de uma dependência funcional não trivial é chave candidata; portanto não há dependência transitiva nem determinante que não seja chave.

5.1 Análise específica de Animal

Raça e espécie. A hipótese de que raça → espécie (por exemplo, “Persa” só existe em gatos) não vale como dependência funcional do domínio: o valor “SRD” (sem raça definida) aparece associado a Cachorro, Gato e Coelho, tanto no modelo quanto na carga. Assim, raca sozinha não determina especie, e o par (especie, raca) é que carrega a informação. Não há, portanto, dependência transitiva entre atributos não-chave, e Animal permanece em 3FN e em BCNF. A coerência entre raça e espécie (por exemplo, impedir “Persa” em cachorro) é uma regra de integridade de domínio, e não uma questão de normalização; ela é tratada na aplicação por lista fechada de combinações válidas no formulário de cadastro. Caso a regra precise ser garantida pelo banco, a evolução seria um catálogo raca(id_raca, nome, especie) referenciado por Animal.

5.2 Desnormalização deliberada: Animal.status_atual

É uma redundância de dados, pois o valor de animal.status_atual pode ser obtido a partir da tabela situacao_animal, considerando o registro mais recente

de cada animal_id. Mesmo assim, optou-se por manter status_atual como uma coluna própria na tabela animal.

Foi escolhido intencionalmente manter essa redundância porque verifica se um animal está "Disponível" é uma consulta muito frequente no sistema, sendo utilizada nas telas de adoção e na validação da RN10, na qual o pedido só pode ser aprovado se o animal estiver disponível ou em adoção. Sem a coluna redundante, cada uma dessas consultas exigiria buscar o registro de maior data_registro para cada animal_id na tabela situacao_animal, tornando a consulta mais trabalhosa e custosa. Por isso, a atualização de status_atual é realizada sempre que um novo registro de situação é inserido, conforme a RN12, mantendo os dados sincronizados e reduzindo o risco de inconsistências.

5.3 Análise específica de Auditoria

Os valores guardados como cópia não são redundâncias indevidas, porque cada linha é um fato histórico e não pode depender do valor atual da tabela original, a justificativas delas são:

- **Repetição dos dados do evento**. Quando um único UPDATE altera vários campos, as linhas geradas repetem tabela, id_registro, autor e data_hora. Existe uma entidade implícita, o "evento de alteração", que poderia ser separada em duas tabelas (cabeçalho e detalhe). Isso não viola a BCNF, porque a relação não tem um identificador do evento, e portanto nenhuma dependência desse tipo existe nos atributos dela. Mantivemos uma tabela só pelos seguintes motivos:
    - Cada gatilho audita no máximo 2 ou 3 campos, então a economia de espaço seria pequena.
    - Cada alteração exigiria um segundo INSERT e a recuperação do identificador do evento dentro do gatilho.
    - A tela de histórico faz uma consulta simples, sem junção.

- **Valores copiados**. O campo valor_posterior de uma alteração é igual ao valor_anterior da seguinte, e o último valor_posterior é igual ao valor atual na tabela de origem. Esse tipo de repetição entre linhas não é dependência funcional. Ela é intencional: cada linha precisa ser autossuficiente como evidência, mesmo que as demais sejam arquivadas. Ela também permite detectar lacunas, porque um valor_anterior que não bate com o valor_posterior da linha anterior indica alteração fora do registro.

- **Referência sem chave estrangeira**. Os campos tabela, id_registro aponta para tabelas diferentes, o que uma FK não consegue expressar, e uma FK com exclusão em cascata apagaria o histórico junto com o registro. O custo é a perda da garantia de integridade referencial pelo banco.
  Ela é compensada pelo próprio gatilho que grava, a partir de OLD.id_animal ou OLD.id_pedido, então o id_registro é sempre válido no momento da gravação.
  A alternativa de ter uma tabela de auditoria por tabela auditada preservaria a FK, mas duplicaria a estrutura a cada nova tabela auditada.

ETAPA 2 - Operação, interface e desempenho

B3 Transações e controle de concorrência

- **B3.1 Operação transacional: aprovação de pedido de adoção**

A aprovação de um pedido de adoção altera três tabelas (pedidoAdocao, Animal e SituacaoAnimal) e por isso é executada em uma única transação. A sequência de comandos da aplicação é:

```sql
START TRANSACTION;

-- 1) Trava, numa única instrução, a linha do pedido e a linha do animal
SELECT p.status_pedido, a.id_animal, a.status_atual, a.id_ong
  FROM pedidoAdocao p
  JOIN Animal a ON a.id_animal = p.id_animal
 WHERE p.id_pedido = ?
   FOR UPDATE;

-- 2) A aplicação reconfere as regras: pedido 'Em análise' (RN06), animal
--    'Disponível' ou 'Em Adoção' (RN10) e avaliador da mesma ONG (RN21).
--    Se alguma falhar: ROLLBACK e mensagem ao usuário.

-- 3) Altera as três tabelas
UPDATE pedidoAdocao
   SET status_pedido = 'Aprovado', id_adm_ong = ?, data_decisao = NOW(), parecer = ?
 WHERE id_pedido = ?;                                    -- RN06, RN09

UPDATE Animal SET status_atual = 'Adotado' WHERE id_animal = ?;

UPDATE SituacaoAnimal SET data_fim = CURDATE()          -- RN12: encerra o registro vigente
 WHERE id_animal = ? AND data_fim IS NULL;

SELECT COALESCE(MAX(registro), 0) + 1 FROM SituacaoAnimal WHERE id_animal = ?;

INSERT INTO SituacaoAnimal (id_animal, registro, data_registro, motivo, status, data_inicio)
VALUES (?, <registro calculado>, NOW(), 'Adoção aprovada (pedido #?)', 'Adotado', CURDATE());
```

```sql
28
29      -- 4) Os demais pedidos pendentes do mesmo animal são recusados automaticamente
30      UPDATE pedidoAdocao
31         SET status_pedido = 'Recusado', id_adm_ong = ?, data_decisao = NOW(),
32             parecer = 'Animal adotado em outro pedido'
33       WHERE id_animal = ? AND id_pedido <> ? AND status_pedido = 'Em análise';
34
35      COMMIT;       -- sucesso: tudo é gravado junto
36      -- ROLLBACK; -- qualquer erro nos passos acima: nada é gravado
```

- **Demonstração de ROLLBACK**

Histórico de situação da Pandora (antes)



Figura B3.1: Estado inicial do animal Pandora antes da operação. Histórico com 3 registros e situação vigente ’Em adoção’

Tela “Avaliar pedido #35” com a mensagem vermelha



Figura B3.2: Falha provocada propositalmente durante a aprovação do pedido #35. A interface informa que o ROLLBACK desfez todas as alterações.

Histórico da Pandora (depois do rollback)

Idêntico ao primeiro: continua com 3 registros e “Em adoção” vigente. É a prova de integridade.



Histórico de situação: Pandora

Cada mudança de situação encerra o registro aberto e cria um novo (RN12).

REGISTRO	REGISTRADO EM	SITUAÇÃO	MOTIVO	INÍCIO	FIM
1	2026-04-03 08:00:00	TRATAMENTO	Resgate e acolhimento	2026-04-03	2026-04-18
2	2026-04-18 08:00:00	DISPONÍVEL	Alta médica, liberado para adoção	2026-04-18	2026-05-05
3	2026-05-05 08:00:00	EM ADOÇÃO	Pedido de adoção em avaliação	2026-05-05	—

3 linha(s).

← Voltar aos animais

Figura B3.3: Histórico da Pandora após o ROLLBACK, sem alteração. Nenhum registro novo foi criado e o registro 3 continua aberto.

“Últimas decisões” com o pedido #35 aprovado



Últimas decisões

PEDIDO	DECIDIDO EM	ANIMAL	SITUAÇÃO DO ANIMAL	ADOTANTE	DECISÃO	PARECER
#35	2026-10-15 09:17:31	P Pandora	ADOTADO	Marcos Vinícius Braga	APROVADO	—

Figura B3.4: Desfecho de sucesso. Na segunda tentativa, sem a falha, a transação foi concluída (COMMIT): pedido aprovado e animal Adotado.

Histórico final de Pandora depois do COMMIT



Histórico de situação: Pandora

Cada mudança de situação encerra o registro aberto e cria um novo (RN12).

REGISTRO	REGISTRADO EM	SITUAÇÃO	MOTIVO	INÍCIO	FIM
1	2026-04-03 08:00:00	TRATAMENTO	Resgate e acolhimento	2026-04-03	2026-04-18
2	2026-04-18 08:00:00	DISPONÍVEL	Alta médica, liberado para adoção	2026-04-18	2026-05-05
3	2026-05-05 08:00:00	EM ADOÇÃO	Pedido de adoção em avaliação	2026-05-05	2026-10-15
4	2026-10-15 09:17:31	ADOTADO	Adoção aprovada (pedido #35)	2026-10-15	—

4 linha(s).

Figura B3.5: Histórico de Pandora após o COMMIT. O registro 3 foi encerrado e o registro 4 (‘Adotado’) foi criado, mostrando que pedido, animal e histórico foram alterados juntos.

- **Parte 2: Experimento com duas sessões (no Workbench)**

Figura B3.6: Sessão A (08:47:30): inicia a transação e executa `SELECT ... FOR UPDATE` no pedido #40. **Status lido: "Em análise".** O lock é adquirido.

```
1  USE petcare_db;
2  SELECT id_pedido, status_pedido FROM pedidoAdocao WHERE status_pedido = 'Em análise';
3
4  START TRANSACTION;
5  SELECT id_pedido, status_pedido FROM pedidoAdocao WHERE id_pedido = 40 FOR UPDATE;
6
7  UPDATE pedidoAdocao SET status_pedido = 'Aprovado', data_decisao = NOW() WHERE id_pedido = 40;
8  COMMIT;
9
10
```

Result Grid | Filter Rows: | Edit: | Export/Import: | Wrap Cell Content:

id_pedido	status_pedido
40	Em análise
NULL	NULL

pedidoAdocao 2

Output

Action Output

#	Time	Action	Message
1	08:47:22	USE petcare_db	0 row(s) affected
2	08:47:22	SELECT id_pedido, status_pedido FROM pedidoAdocao WHERE status_pedido = 'Em análise' LIMIT 0, 1000	6 row(s) returned
3	08:47:30	START TRANSACTION	0 row(s) affected
4	08:47:30	SELECT id_pedido, status_pedido FROM pedidoAdocao WHERE id_pedido = 40 LIMIT 0, 1000 FOR UPDATE	1 row(s) returned

Figura B3.7: Sessão B (08:47:59): tenta o mesmo `SELECT ... FOR UPDATE` e fica em **"Running..." (bloqueada)**, aguardando a Sessão A.

```
3  START TRANSACTION;
4  SELECT id_pedido, status_pedido FROM pedidoAdocao WHERE id_pedido = 40 FOR UPDATE;
5
6  ROLLBACK;
```

Output

Action Output

#	Time	Action	Message
1	08:47:54	USE petcare_db	0 row(s) affected
2	08:47:59	START TRANSACTION	0 row(s) affected
3	08:47:59	SELECT id_pedido, status_pedido FROM pedidoAdocao WHERE id_pedido = 40 LIMIT 0, 1000 FOR UPDATE	Running...

Figura B3.8: Sessão A (08:48:08): *executa o* `UPDATE` ***(status "Aprovado")*** *e o* `COMMIT`*, liberando o lock.*

```
7 •  UPDATE pedidoAdocao SET status_pedido = 'Aprovado', data_decisao = NOW() WHERE id_pedido = 40;
8 •  COMMIT;
```

Output

Action Output

#	Time	Action	Message
1	08:47:22	USE petcare_db	0 row(s) affected
2	08:47:22	SELECT id_pedido, status_pedido FROM pedidoAdocao WHERE status_pedido = 'Em análise' LIMIT 0, 1000	6 row(s) returned
3	08:47:30	START TRANSACTION	0 row(s) affected
4	08:47:30	SELECT id_pedido, status_pedido FROM pedidoAdocao WHERE id_pedido = 40 LIMIT 0, 1000 FOR UPDATE	1 row(s) returned
5	08:48:08	UPDATE pedidoAdocao SET status_pedido = 'Aprovado', data_decisao = NOW() WHERE id_pedido = 40	1 row(s) affected Rows matched: 1 Changed: 1 Warnings: 0
6	08:48:08	COMMIT	0 row(s) affected

Figura B3.9: Sessão B é liberada após o COMMIT da A e lê o status atualizado **"Aprovado".**

```
1 •  USE petcare_db;
2
3 •  START TRANSACTION;
4 •  SELECT id_pedido, status_pedido FROM pedidoAdocao WHERE id_pedido = 40 FOR UPDATE;
5
6 •  ROLLBACK;
```

Result Grid | Filter Rows: | Edit: | Export/Import: | Wrap Cell Content:

id_pedido	status_pedido
40	Aprovado
NULL	NULL

edidoAdocao 1 ×

Output

Action Output

#	Time	Action	Message
1	08:47:54	USE petcare_db	0 row(s) affe
2	08:47:59	START TRANSACTION	0 row(s) affe
3	08:47:59	SELECT id_pedido, status_pedido FROM pedidoAdocao WHERE id_pedido = 40 LIMIT 0, 1000 FOR UPDATE	1 row(s) retu

Figura B3.10: Nível de isolamento em vigor nas Sessões A e B: **REPEATABLE-READ** (padrão do InnoDB).

Sessão A

```
10 •  SELECT @@transaction_isolation;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

@@transaction_isolation
REPEATABLE-READ

Result 3

Output

Action Output

#	Time	Action
1	08:47:22	USE petcare_db
2	08:47:22	SELECT id_pedido, status_pedido FROM pedidoAdocao WHERE status_pedido = 'Em análise' LIMIT 0, 1000
3	08:47:30	START TRANSACTION
4	08:47:30	SELECT id_pedido, status_pedido FROM pedidoAdocao WHERE id_pedido = 40 LIMIT 0, 1000 FOR UPDATE
5	08:48:08	UPDATE pedidoAdocao SET status_pedido = 'Aprovado', data_decisao = NOW() WHERE id_pedido = 40
6	08:48:08	COMMIT
7	08:50:37	SELECT @@transaction_isolation LIMIT 0, 1000

Sessão B

```
1 •  USE petcare_db;
2
3 •  START TRANSACTION;
4 •  SELECT id_pedido, status_pedido FROM pedidoAdocao WHERE id_pedido = 40 FOR UPDATE;
5
6 •  ROLLBACK;
7
8 •  SELECT @@transaction_isolation;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

@@transaction_isolation
REPEATABLE-READ

Result 2

Output

Action Output

#	Time	Action
1	08:47:54	USE petcare_db
2	08:47:59	START TRANSACTION
3	08:47:59	SELECT id_pedido, status_pedido FROM pedidoAdocao WHERE id_pedido = 40 LIMIT 0, 1000 FOR UPDATE
4	08:50:03	SELECT @@transaction_isolation LIMIT 0, 1000

Observação: A Sessão A adquiriu o lock do pedido #40 às 08:47:30. A Sessão B tentou o mesmo SELECT ... FOR UPDATE às 08:47:59 e permaneceu bloqueada (Running...) até o COMMIT da Sessão A, às 08:48:08, ou seja, cerca de 9 segundos de espera. Ao ser liberada, a B leu o status “Aprovado”, e não “Em análise”. Não houve

deadlock; o comportamento observado foi de espera por bloqueio, sob o nível REPEATABLE-READ.

O UPDATE da Sessão A foi executado apenas para demonstrar o bloqueio: ele altera somente o pedido. Depois do experimento, o pedido #40 foi restaurado para “Em análise”, mantendo a base consistente

Justificativa do nível de isolamento adotado

A aplicação utiliza o nível padrão do InnoDB, **REPEATABLE READ** (Figura B3.10). Nesse nível, leituras comuns dentro de uma transação enxergam um *snapshot* consistente. Porém, o snapshot sozinho não impede que duas sessões decidam o mesmo pedido. Por isso, a aprovação usa um único `SELECT ... FOR UPDATE` sobre o pedido e o animal. Esse tipo de leitura com bloqueio não usa o snapshot: ela retorna o dado mais recente já confirmado.

O experimento (Figuras B3.6 a B3.9) mostra o efeito: a Sessão B ficou bloqueada por cerca de 9 segundos até o COMMIT da Sessão A e, ao ser liberada, leu o status “Aprovado”. Na aplicação, após obter os bloqueios, a transação reconfere se o pedido continua “Em análise” e se o animal está “Disponível” ou “Em Adoção” (RN10); caso contrário, executa ROLLBACK e informa o usuário.

Isso evita a aprovação duplicada do mesmo animal por pedidos diferentes. Evita também registros de histórico duplicados (RN12), pois o cálculo de registro = MAX(registro)+1 ocorre com a linha do animal bloqueada. Como proteção adicional, a chave primária (id_animal, registro) de SituacaoAnimal faria o banco rejeitar um registro duplicado, ainda que o bloqueio falhasse.

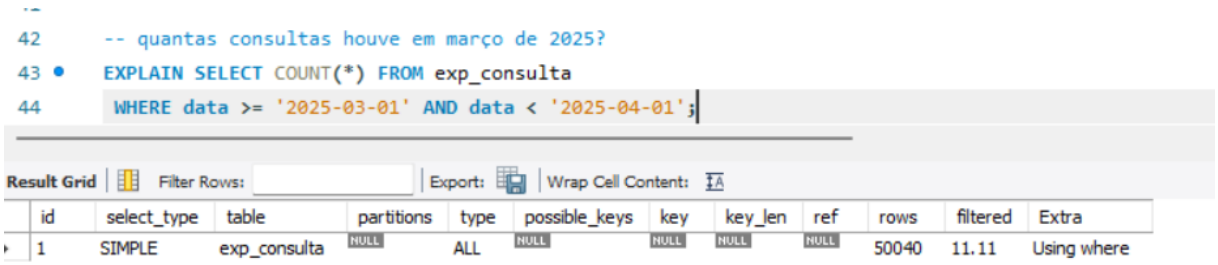
O nível SERIALIZABLE foi descartado porque faria o InnoDB aplicar bloqueios compartilhados também às leituras comuns, aumentando esperas e o risco de deadlock nas telas de consulta, sem benefício adicional, já que o bloqueio explícito cobre a operação crítica. Para reduzir o risco de deadlock, os dois bloqueios (pedido e animal) são adquiridos na mesma instrução, em todas as execuções, e a transação é mantida curta.

B4 Otimização de consultas

As tabelas do projeto têm poucas dezenas de linhas, volume em que o otimizador ignora índices e lê a tabela inteira. Para tornar o efeito mensurável, os estudos usaram duas tabelas de experimento, exp_consulta e exp_animal, com 50.000 linhas cada, criadas à parte e sem relação com as tabelas reais. Em cada estudo comparamos o EXPLAIN antes e depois de criar o índice, observando as colunas type, key, rows e filtered. As tabelas de experimento foram removidas ao final.

B4.1 Estudo 1: filtro por período (índice simples)

Antes (sem índice):

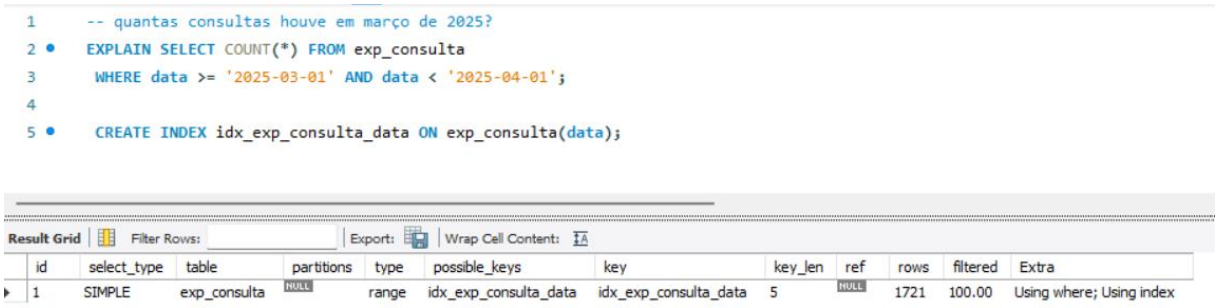


```
42    -- quantas consultas houve em março de 2025?
43 •  EXPLAIN SELECT COUNT(*) FROM exp_consulta
44     WHERE data >= '2025-03-01' AND data < '2025-04-01';
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

id	select_type	table	partitions	type	possible_keys	key	key_len	ref	rows	filtered	Extra
1	SIMPLE	exp_consulta	NULL	ALL	NULL	NULL	NULL	NULL	50040	11.11	Using where

Depois (com índice)



```
1    -- quantas consultas houve em março de 2025?
2 •  EXPLAIN SELECT COUNT(*) FROM exp_consulta
3     WHERE data >= '2025-03-01' AND data < '2025-04-01';
4
5 •   CREATE INDEX idx_exp_consulta_data ON exp_consulta(data);
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

id	select_type	table	partitions	type	possible_keys	key	key_len	ref	rows	filtered	Extra
1	SIMPLE	exp_consulta	NULL	range	idx_exp_consulta_data	idx_exp_consulta_data	5	NULL	1721	100.00	Using where; Using index

Sem índice em `data`, o MySQL leu a tabela inteira (`type = ALL`, ~50 mil linhas estimadas) para filtrar um mês. Com `idx_consulta_data`, passou a usar uma varredura por faixa (`type = range`) e estima ler ~1.721 linhas, e a coluna `filtered` subiu de 11,11% para 100%. O filtro por período é usado na consulta de atendimentos, então o índice é justificado.

B4.2 Estudo 2: filtro combinado (índice simples contra composto)

Pergunta: animais “Disponível” da espécie “Gato”.

Antes do índice

```
8   -- animais "Disponível" da espécie "Gato"
9   EXPLAIN SELECT id_animal, nome FROM exp_animal
10  WHERE status_atual = 'Disponível' AND especie = 'Gato';
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

id	select_type	table	partitions	type	possible_keys	key	key_len	ref	rows	filtered	Extra
1	SIMPLE	exp_animal	NULL	ALL	NULL	NULL	NULL	NULL	50274	1.00	Using where

Índice simples:

```
8   -- animais "Disponível" da espécie "Gato"
9   EXPLAIN SELECT id_animal, nome FROM exp_animal
10  WHERE status_atual = 'Disponível' AND especie = 'Gato';
11
12  CREATE INDEX idx_exp_animal_status ON exp_animal(status_atual);
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

id	select_type	table	partitions	type	possible_keys	key	key_len	ref	rows	filtered	Extra
1	SIMPLE	exp_animal	NULL	ref	idx_exp_animal_status	idx_exp_animal_status	202	const	2631	10.00	Using where

Índice composto:

```
8   -- animais "Disponível" da espécie "Gato"
9   EXPLAIN SELECT id_animal, nome FROM exp_animal
10  WHERE status_atual = 'Disponível' AND especie = 'Gato';
11
12  CREATE INDEX idx_exp_animal_status_especie ON exp_animal(status_atual, especie);
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

id	select_type	table	partitions	type	possible_keys	key	key_len	ref	rows	filtered	Extra
1	SIMPLE	exp_animal	NULL	ref	idx_exp_animal_status,idx_exp_animal_status_...	idx_exp_animal_status_especie	404	const,const	658	100.00	NULL

O índice simples em `status_atual` reduziu a leitura de ~50 mil para ~2.600 linhas, mas ainda precisou filtrar a espécie linha a linha. O índice composto (`status_atual, especie`) resolveu os dois filtros dentro do índice e caiu para ~650 linhas. A ordem das colunas importa: `status_atual` vem primeiro porque o índice composto também atende buscas só por status (prefixo mais à esquerda) . A coluna `filtered`, que estima o percentual de linhas lidas que realmente atendem ao filtro, subiu de 1% (sem índice) para 10% (índice simples) e para 100% (índice composto).

B4.3 Estudo 3: quando o índice não ajuda (LIKE)

Pergunta: a busca por nome da tela de Animais usa `LIKE`. O índice ajuda nos dois jeitos de escrever?

Antes do índice

```
14    -- A busca por nome da tela de Animais usa LIKE. O índice ajuda nos dois jeitos de escrever?
15 •  EXPLAIN SELECT id_animal, nome, especie FROM exp_animal WHERE nome LIKE 'Bob1%';
16
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

id	select_type	table	partitions	type	possible_keys	key	key_len	ref	rows	filtered	Extra
1	SIMPLE	exp_animal	NULL	ALL	NULL	NULL	NULL	NULL	50274	11.11	Using where

Depois (com índice)

```
14    -- A busca por nome da tela de Animais usa LIKE. O índice ajuda nos dois jeitos de escrever?
15 •  EXPLAIN SELECT id_animal, nome, especie FROM exp_animal WHERE nome LIKE 'Bob1%';
16
17 •  CREATE INDEX idx_exp_animal_nome ON exp_animal(nome);
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

id	select_type	table	partitions	type	possible_keys	key	key_len	ref	rows	filtered	Extra
1	SIMPLE	exp_animal	NULL	range	idx_exp_animal_nome	idx_exp_animal_nome	402	NULL	1388	100.00	Using index condition

Com curinga no começo ('%ob1%'):

```
17 •  EXPLAIN SELECT id_animal, nome, especie FROM exp_animal WHERE nome LIKE '%ob1%';
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

id	select_type	table	partitions	type	possible_keys	key	key_len	ref	rows	filtered	Extra
1	SIMPLE	exp_animal	NULL	ALL	NULL	NULL	NULL	NULL	50274	11.11	Using where

O índice em nome só é usado quando a busca é por prefixo (LIKE 'Bob1%', ~1.388 linhas). Com LIKE '%ob1%', o MySQL precisa examinar todas as linhas (type = ALL, ~50 mil), porque não sabe em que posição do índice começar. A busca por nome da tela de Animais usa o padrão %termo%, então este índice **não** traria ganho para ela e, por isso, não foi adotado. Este estudo mostra o limite do índice: um índice só vale quando a forma como a consulta é escrita permite usá-lo.

B4.4 Índices adotados

Com base nos estudos, foram adicionados ao banco idx_consulta_data em Consulta(data) e idx_animal_status_especie em Animal(status_atual, especie), criados no script 05_indices.sql. O índice em nome não foi adotado (Estudo 3). Todo índice tem um custo: ocupa espaço e deixa INSERT e UPDATE um pouco mais lentos, por isso só foram criados os que atendem a filtr*os usados pela apli*cação.

Índices criados no banco real

- idx_animal_status_especie

```
7 • SHOW INDEX FROM Animal;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

Table	Non_unique	Key_name	Seq_in_index	Column_name	Collation	Cardinality	Sub_part	Packed	Null	Index_typ
animal	0	PRIMARY	1	id_animal	A	40	NULL	NULL		BTREE
animal	1	idx_animal_ong	1	id_ong	A	3	NULL	NULL		BTREE
animal	1	idx_animal_status_especie	1	status_atual	A	5	NULL	NULL		BTREE
animal	1	idx_animal_status_especie	2	especie	A	11	NULL	NULL		BTREE

- idx_consulta_data

```
7 • SHOW INDEX FROM Consulta;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

Table	Non_unique	Key_name	Seq_in_index	Column_name	Collation	Cardinality	Sub_part	Packed	Null	Index_t
consulta	0	PRIMARY	1	id_consulta	A	70	NULL	NULL		BTREE
consulta	1	fk_consulta_prontuario	1	id_prontuario	A	40	NULL	NULL		BTREE
consulta	1	idx_consulta_veterinario	1	id_veterinario	A	4	NULL	NULL		BTREE
consulta	1	idx_consulta_data	1	data	A	70	NULL	NULL		BTREE

B5 Segurança e controle de acesso

1. Objetivo

Esta seção apresenta a parte B5 do projeto Instituto PetCare. Ela mostra como os privilégios de acesso foram separados por função, como visões limitam os dados que cada perfil enxerga, como as credenciais são armazenadas e quais cuidados foram tomados com os dados pessoais. A implementação corresponde ao script de segurança do projeto (05_seguranca.controle.sql).

2. Perfis de acesso e princípio do menor privilégio [B5.1]

Foram criados quatro usuários no banco de dados, acima do mínimo de três pedido no enunciado. Cada conta recebe apenas as operações de que precisa para a sua função.

Perfil	Finalidade	Privilégios previstos
petcare_admbd	Administração do banco	ALL PRIVILEGES no esquema petcare_db; responsável por manutenção e administração.
petcare_app	Operações da aplicação	Privilégios de leitura e escrita concedidos por tabela; sem privilégios de DDL e sem acesso direto à tabela CredencialAcesso ou à tabela Auditoria.
petcare_auditor	Auditoria e consulta	Leitura da tabela Auditoria e de visões/tabelas específicas; sem permissões de escrita.
petcare_publico	Visitante externo	SELECT somente na visão vw_animais_disponiveis.

O script usa GRANT para conceder privilégios e REVOKE para retirá-los. No perfil petcare_app, o DELETE é revogado das tabelas que guardam dados e históricos importantes, como Pessoa, Animal, Prontuario, Consulta e PedidoAdocao. O DELETE continua disponível apenas em algumas tabelas de cadastro ou catálogo.

Com isso aplicamos o princípio do menor privilégio: cada usuário só tem as permissões necessárias para a sua atividade. Os comandos SHOW GRANTS permitem conferir os privilégios atribuídos a cada conta.

3. Visões para restrição de dados [B5.2]

O script cria três visões para limitar os dados apresentados a determinados perfis. As visões funcionam como uma
camada de acesso que expõe somente as informações selecionadas em cada consulta.

Visão	Perfil autorizado	Restrição aplicada
vw_animais_disponiveis	petcare_publico	Exibe animais cujo status_atual é 'Disponível' e seleciona colunas destinadas à consulta pública, sem expor o id_ong nem o status interno.
vw_pedidos_auditoria	petcare_auditor	Apresenta informações de pedidos necessárias à conferência, sem incluir diretamente nome, CPF, endereço ou telefone do adotante.
vw_credenciais_auditoria	petcare_auditor	Exibe id_pessoa, login e data_cadastro; não expõe as colunas salt e senha_hash.

O perfil petcare_publico só pode consultar a visão vw_animais_disponiveis e não tem acesso às tabelas-base. O auditor acessa as visões de auditoria e as tabelas autorizadas no script, sem ver as colunas pessoais nem as de sal e hash. Assim, cada perfil recebe só as informações de que precisa.

4. Armazenamento de credenciais com hash e sal [B5.3]

No procedimento sp_cadastrar_credencial, o banco verifica se a senha tem pelo menos oito caracteres, gera um sal aleatório com RANDOM_BYTES(16), converte o valor para hexadecimal e calcula o hash com SHA2(CONCAT(sal, senha), 256). O resultado vai para a coluna senha_hash e o sal fica separado, na coluna salt.
No procedimento sp_autenticar_credencial, o hash é recalculado com o sal guardado e a senha informada, e o resultado é comparado com o valor gravado. Se a combinação estiver correta, o procedimento devolve o identificador e o login; caso contrário, não devolve nenhuma linha.

A aplicação recebe permissão para executar os dois procedimentos, mas não tem privilégio direto sobre CredencialAcesso. Portanto, o acesso às credenciais passa sempre por eles.

O SHA2 com sal atende ao que foi pedido e mostra o conceito. Em um sistema real, o ideal seria usar um algoritmo de hash de senha mais lento, como bcrypt ou Argon2, além de proteger a conexão com o banco.

5. Dados pessoais e cuidados adotados [B5.4]

São dados pessoais as informações que identificam ou podem ser associadas a uma pessoa. No PetCare, são eles:

- Pessoa e subtipos (Adotante, Veterinario, AdmOng e AdmSistema): CPF, nome, data de nascimento, endereço e telefone.
- Parceiro: nome e documento (CPF ou CNPJ).
- CredencialAcesso: login e identificador da pessoa. O sal e o hash não são a senha, mas continuam protegidos por serem dados de autenticação.
- Pedido de adoção: dados que ligam o pedido a um adotante, como datas, decisões e pareceres.

Os cuidados adotados foram:

- limitar os privilégios por perfil;
- restringir a consulta pública à visão de animais disponíveis;
- dar ao auditor visões que omitem colunas pessoais e as colunas de sal e hash;
- impedir que a aplicação consulte CredencialAcesso diretamente;
- usar REVOKE DELETE nas tabelas com dados e históricos importantes.

Essas medidas reduzem a exposição e a alteração indevida dos dados, mas sozinhas não garantem a conformidade com a LGPD. Isso também depende da finalidade do tratamento, da base legal e da política de retenção.

6. Testes de acesso

Testamos o acesso com cada usuário:

- o visitante consulta a visão pública, mas recebe erro de permissão ao consultar a tabela Pessoa;
- o usuário da aplicação recebe erro ao consultar CredencialAcesso diretamente e ao executar DELETE nas tabelas em que ele foi revogado;
- a autenticação com a senha correta devolve o id e o login, e com a senha errada não devolve nenhuma linha;
- o auditor consulta as visões autorizadas, mas recebe erro ao tentar atualizar qualquer tabela.

7. Conclusão

A parte B5 organiza o controle de acesso do PetCare em quatro contas com funções distintas, com concessão e revogação de privilégios, três visões de restrição e procedimentos de cadastro e autenticação com hash e sal. Também reúne os cuidados para reduzir a exposição de dados pessoais e preservar os registros importantes.

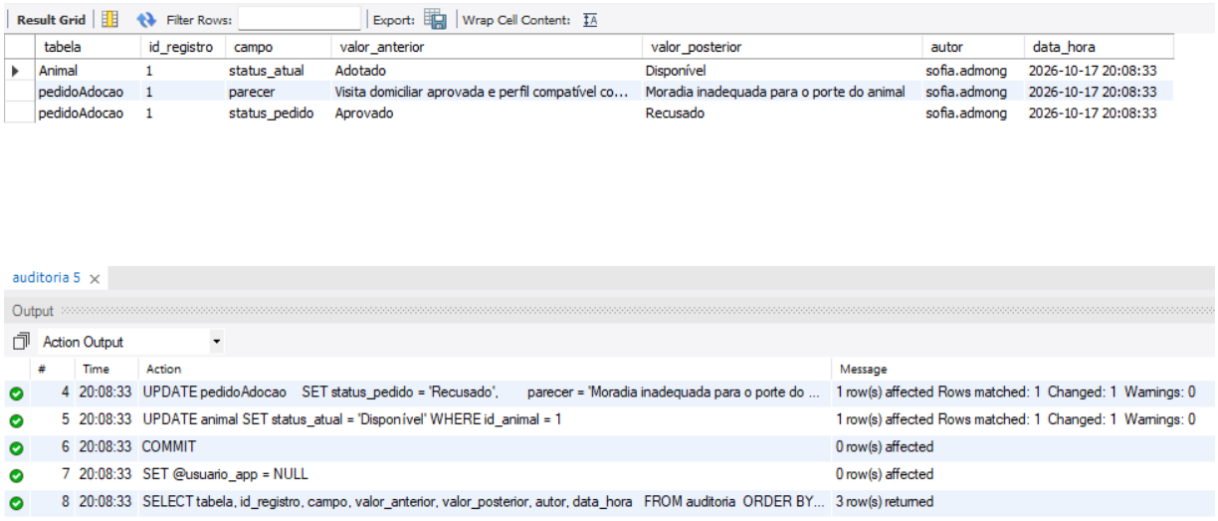
B6 Gatilhos e auditoria

Gatilho	Tabela/Campos	Justificativa de domínio
trg_animal_audita_update	Animal → status_atual	O status define se o animal pode ser adotado (RN10), e a RN23 diz que animal adotado nunca é excluído, só muda de status. Saber quem marcou um animal como "Adotado" ou "Óbito" e quando, é essencial para a rastreabilidade.
trg_pedido_audita_update	PedidoAdocao → status_pedido, parecer, id_adm_ong	É a decisão mais sensível do sistema: aprovar ou recusar uma adoção define o destino de um animal. A RN08 exige parecer na recusa, e a RN21 diz que só o admOng decide. A auditoria prova quem

		decidiu, quando, e comprova que o parecer não foi alterado depois.

O que não foi auditado:

- Animal.descricao, nome e raca mudam com frequência, não têm efeito jurídico nem operacional e gerariam ruído e custo de armazenamento.
- SituacaoAnimal já é um histórico datado por si só (RN12), então auditar a tabela seria redundante.



Result Grid | Filter Rows: | Export: | Wrap Cell Content:

tabela	id_registro	campo	valor_anterior	valor_posterior	autor	data_hora
Animal	1	status_atual	Adotado	Disponível	sofia.admong	2026-10-17 20:08:33
pedidoAdocao	1	parecer	Visita domiciliar aprovada e perfil compatível co...	Moradia inadequada para o porte do animal	sofia.admong	2026-10-17 20:08:33
pedidoAdocao	1	status_pedido	Aprovado	Recusado	sofia.admong	2026-10-17 20:08:33

auditoria 5

Output

Action Output

#	Time	Action	Message
4	20:08:33	UPDATE pedidoAdocao SET status_pedido = 'Recusado', parecer = 'Moradia inadequada para o porte do ...	1 row(s) affected Rows matched: 1 Changed: 1 Warnings: 0
5	20:08:33	UPDATE animal SET status_atual = 'Disponível' WHERE id_animal = 1	1 row(s) affected Rows matched: 1 Changed: 1 Warnings: 0
6	20:08:33	COMMIT	0 row(s) affected
7	20:08:33	SET @usuario_app = NULL	0 row(s) affected
8	20:08:33	SELECT tabela, id_registro, campo, valor_anterior, valor_posterior, autor, data_hora FROM auditoria ORDER BY...	3 row(s) returned

Figura B.1: Teste da tabela auditoria por meio de uma transação.

Observação: Como neste projeto não foi implementado o formulário, para a execução deste teste na interface, colocamos no próprio formulário a escolha do usuário para ocupar o campo de autor da tabela somente para facilitar a apresentação do projeto.

B7 Projeto físico e armazenamento

Esta seção documenta as decisões de projeto físico do PetCare no MySQL 8.0.46 (InnoDB), retomando o conteúdo das aulas 12, 13 e 15.

B7.1 Revisão dos tipos de dados

Revisamos os tipos do modelo lógico e mudaríamos o seguinte:

- Vacina.num_dose (CHAR(5)) para TINYINT UNSIGNED, porque é um número. Como texto, aceita valores inválidos e ordena errado ('10' vem antes de '2').

- AplicacaoVacina.lote (CHAR(20)) e Veterinario.crmv (CHAR(15)) para VARCHAR, porque o tamanho varia e o CHAR desperdiça espaço com preenchimento.

- Listas fechadas (status do animal, status do pedido, sexo etc.), hoje controladas só pela aplicação (RN26), passariam a ter CHECK no banco. Assim um INSERT direto também não aceita valor inválido.

- SolicitacaoExame.resultado (VARCHAR(500)) para TEXT, porque laudos podem passar de 500 caracteres.

Mantivemos DECIMAL(10,2) nos valores de patrocínio (é exato), DATETIME em vez de TIMESTAMP e INT nas chaves substitutas.

B7.2 Estimativa de crescimento em dois anos

Hoje o banco tem poucas dezenas de linhas. Para projetar, supomos 20 ONGs acolhendo cerca de 120 animais por ano cada, ou seja, ~5.000 animais em dois anos. Com a média de 4 consultas, 3,5 registros de situação e 3 vacinas por animal, as tabelas de maior movimento chegam a:

- Consulta: ~20.000 linhas (~5 MB)

- SituacaoAnimal: ~17.500 linhas (~3,5 MB)

- AplicacaoVacina: ~15.000 linhas (~1 MB)

- auditoria: ~30.000 linhas (~7,5 MB)

No total, o banco chega a cerca de 30 MB com índices. É pequeno e cabe inteiro na memória do InnoDB, então não é preciso mais de um servidor.

B7.3 Plano de backup

- O que copiar: o banco petcare_db inteiro (tabelas, dados, gatilhos e visão) e os binary logs.

- Frequência: backup completo por mysqldump --single-transaction todo dia de madrugada, e cópia dos binary logs a cada hora.

- Retenção: 7 backups diários e 4 semanais, guardados em duas mídias, uma fora do servidor, criptografados. Como têm CPF e endereço, não são mantidos por mais de 6 meses (LGPD).

- Teste de restauração: uma vez por mês, o backup mais recente é restaurado em uma base separada, e comparamos a contagem de linhas de cada tabela e o resultado das 15 consultas de 03_consultas.sql com os da base original.

B7.4 Particionamento e arquivamento

Particionamento não foi adotado. O volume é pequeno, e o InnoDB não aceita chave estrangeira em tabela particionada, o que quebraria as FKs de Consulta, AplicacaoVacina e SolicitacaoExame.

Arquivamento é proposto só para a tabela auditoria, que apenas recebe inserções, cresce mais rápido e quase não é consultada. Registros com mais de 24 meses seriam movidos para uma tabela auditoria_arquivo, em uma única transação. Os dados de animais, pedidos e prontuários não são arquivados, porque a RN23 exige rastreabilidade.

Declaração de uso de inteligência artificial

Declaração de uso de inteligência artificial

Ferramenta utilizada: Claude (Anthropic)

Em que partes a ferramenta foi utilizada:

1. **Script de carga (02_carga.sql)**: a IA nos orientou na estrutura da carga, incluindo a ordem de inserção que respeita as dependências entre as tabelas e a geração de dados fictícios (nomes, documentos, datas e valores). Nenhum dado pessoal real de terceiros foi usado.
2. **Dicionário de dados conceitual (A3):** a IA ajudou a montar as tabelas de cada entidade no formato do Anexo A, a partir do modelo lógico e do DDL, para evitar divergências.
3. **Revisão de consistência (A1, A3 e 03_consultas.sql):** a IA comparou o dicionário, as regras de negócio, o DDL, a carga e as consultas e apontou divergências, como tamanhos de campo, listas de valores permitidos e o mecanismo que atende cada regra. Ela também sugeriu redações de regras de negócio e ajustes em consultas, que a equipe avaliou e incorporou.
4. **Etapa 2 (B5 e B7):** a IA ajudou a organizar e revisar o texto da parte B5, a partir do script SQL de segurança, e das decisões de projeto físico do B7 (tipos de dados, estimativa de crescimento, backup e arquivamento). A equipe revisou e ajustou o conteúdo.
5. **Relatório final**: a IA ajudou a revisar a consistência do relatório consolidado e a corrigir erros de escrita.